Projet sur la Complexité des Algorithmes de Tri

Université Le Havre Normandie UFR ST

L2 Informatique - Algorithmique et Programmation 4

Chargé de cours : Eric SANLAVILLE

Chargée de TD/TP : Véronique JAY

Auteurs du rapport : Tarzani Yassin et Slamani Ali

Table des matières

1. Introduction.....3

2. Présentation des Tris et de leurs complexités.....3

3. Algorithmes utilisés et comparaison en nombres d'opérations.....4

3.1 Algorithmes utilisés.....4
3.2 Comparaison en nombre de comparaisons.....5
3.3 Les limites de l'optimisation : Les affectations.....6

4. Les algos fonctionnent-ils ?.....6

5. Comparaison en temps d'exécution : mesure en millisecondes.....6

5.1 Description de la mesure.....6
5.2 Le Tri par Insertion : une explosion du temps.....6
5.3 Les Tris Rapides : Fusion vs Dénombrement.....7
5.4 Comparaison globale des trois tris.....7

6. Conclusion.....8

1. Introduction

Comme indiqué dans le nom, on vise au travers de ce projet l'étude de la complexité de différents algorithmes de tri. Dans notre cas, nous nous sommes concentrés sur le Tri par Insertion, le Tri Fusion et le Tri par Dénombrement.

L'objectif est de comparer ces algorithmes de deux façons. D'abord en analysant le nombre d'opérations élémentaires (notamment pour observer l'impact d'une optimisation de recherche dichotomique sur le tri par insertion) , et ensuite en mesurant le temps d'exécution au travers de tests sur des entrées de tailles variables.

2. Présentation des Tris et de leurs complexités

2.1 Présentation des Algorithmes de Tri

Commençons par expliquer le principe de fonctionnement de chacun de nos trois algorithmes de tri.

- **Le Tri par Insertion :** Cet algorithme procède en construisant progressivement un sous-tableau trié. Pour chaque nouvel élément, il cherche sa position correcte dans la partie déjà triée et décale les autres éléments pour l'insérer. Nous avons étudié sa version classique (recherche linéaire) et une version optimisée (recherche dichotomique pour trouver la position plus rapidement).
- **Le Tri Fusion :** Cet algorithme utilise la célèbre technique du "diviser pour régner". Il divise récursivement le tableau en deux moitiés jusqu'à obtenir des sous-tableaux de taille 1, puis il les fusionne de manière ordonnée pour reconstruire le tableau final trié.
- **Le Tri par Dénombrement :** C'est un algorithme non comparatif. Il fonctionne en comptant le nombre d'occurrences de chaque élément (la clé), puis en utilisant ces comptes cumulés pour déterminer la position finale exacte de chaque élément dans le tableau trié.

2.2 Complexité Théorique des Tris

Avant d'analyser nos résultats expérimentaux, rappelons les complexités théoriques de ces algorithmes

- **Tri par Insertion** : Sa complexité temporelle est en $O(n2)$ dans le pire cas et le cas moyen, car il nécessite des parcours et des décalages proportionnels à la taille du tableau. Le meilleur cas (tableau déjà trié) est de $O(n)$.
- **Tri Fusion** : Sa complexité temporelle est garantie à $O(n \log n)$ dans le meilleur, le moyen et le pire cas. C'est un tri très robuste, mais qui nécessite un espace mémoire supplémentaire en $O(n)$ pour les fusions.
- **Tri par Dénombrement** : Sa complexité en temps est strictement linéaire, soit $O(n+k)$ dans tous les cas (où n est le nombre d'éléments et k est la valeur maximale des clés). Cependant, sa complexité spatiale dépend également de k, ce qui peut être très coûteux en mémoire si les valeurs du tableau sont très grandes.

3. Algorithmes utilisés et comparaison en nombres d'opérations

3.1 Algorithmes utilisés

Voici les pseudo-codes simplifiés illustrant la logique de nos implémentations :

Tri par Insertion :

```
Procédure TriInsertion(A, n)
Pour i allant de 1 à n-1 Faire
clé <- A[i]
 j <- i - 1
Tant que j >= 0 et A[j] > clé Faire
A[j + 1] <- A[j]
j <- j - 1
Fin Tant que
    A[j + 1] <- clé
Fin Pour
 Fin Procédure
```

Tri Fusion :

```
Procédure TriFusion(A, début, fin)
Si début < fin Alors
milieu <- (début + fin) / 2
  TriFusion(A, début, milieu)
TriFusion(A, milieu + 1, fin)
   Fusionner(A, début, milieu, fin)
Fin Si
  Fin Procédure
```

Tri par Dénombrement :

```
Procédure TriDenombrement(A, n)
    max <- TrouverMax(A)
    Créer tableau Occurrences de taille max + 1
initialisé à 0
    Créer tableau Sortie de taille n
    Pour i allant de 0 à n-1 Faire
        Occurrences[A[i]] <- Occurrences[A[i]] + 1
    Fin Pour
    Pour i allant de 1 à max Faire
        Occurrences[i] <- Occurrences[i] + Occurrences[i
- 1]
    Fin Pour
    Pour i allant de n-1 à 0 avec un pas de -1 Faire
        Sortie[Occurrences[A[i]] - 1] <- A[i]
        Occurrences[A[i]] <- Occurrences[A[i]] - 1
    Fin Pour
    Copier Sortie dans A
Fin Procédure
```

3.2 Comparaison en nombre de comparaisons

Nous avons voulu mesurer l'impact de l'optimisation dichotomique sur le tri par insertion en comptant le nombre d'opérations.

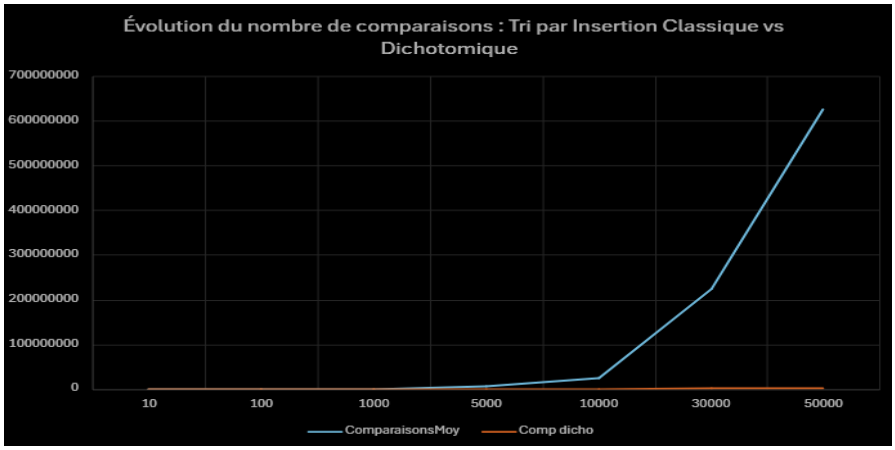

Figure 1: Comparaison du nombre de comparaisons entre le tri par insertion classique et optimisé (dichotomie)

Le graphique ci-dessus met en évidence la différence drastique du nombre de comparaisons effectuées entre la version classique et la version optimisée. Pour le tri par insertion classique, on observe une croissance de forme parabolique typique d'une complexité en O(n^2). À l'inverse, l'optimisation dichotomique écrase totalement cette métrique en utilisant une recherche binaire. La recherche de

position passe d'un coût O(n) à O(log n). Ainsi, pour un tableau de 50 000 éléments, le nombre de comparaisons s'effondre, passant de plus de 625 millions à seulement environ 711 000.

3.3 Les limites de l'optimisation : Les affectations

Si la dichotomie réduit considérablement les comparaisons, notre analyse des compteurs a montré que les affectations (les décalages dans le tableau) restent strictement identiques. En effet, bien que la bonne position d'insertion soit trouvée plus vite, l'algorithme doit toujours procéder au décalage physique de tous les éléments situés à droite pour insérer la clé. C'est pourquoi la complexité temporelle globale du tri reste en O(n^2), nous limitant à des tableaux de 50 000 éléments pour nos tests.

4. Les algos fonctionnent-ils ?

Pour garantir la validité de nos tests expérimentaux, nous avons organisé notre code autour de trois classes distinctes :

- **Element :** Cette classe définit les caractéristiques des éléments que nous allons trier.
- **tabElement :** Elle gère la création et le remplissage aléatoire de nos tableaux de données. Nous y avons intégré une fonction de vérification (un test "estTrie") qui parcourt le tableau pour s'assurer que chaque élément est bien inférieur ou égal au suivant. Cela nous a permis de valider manuellement que nos algorithmes fonctionnaient parfaitement avant de lancer les chronomètres.
- **TestsDesTris :** C'est notre classe principale (le *Main*). Elle est dédiée au lancement automatisé des algorithmes sur des tableaux de tailles variables et à la collecte précise des résultats (temps d'exécution, nombre de comparaisons, nombre d'affectations) pour générer nos graphiques finaux.

5. Comparaison en temps d'exécution : mesure en millisecondes

5.1 Description de la mesure

Pour évaluer les performances temporelles réelles, nous avons généré des tableaux remplis d'éléments aléatoires. Nous avons mesuré le temps moyen d'exécution en millisecondes (ms) pour différentes tailles de tableaux, allant de 10 à 50 000 éléments pour tous les tris, et jusqu'à des tailles bien plus grandes pour les algorithmes les plus performants.

5.2 Le Tri par Insertion : une explosion du temps

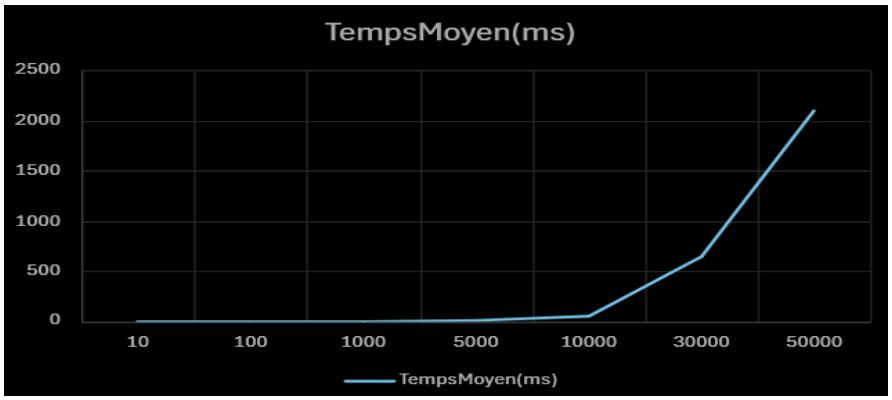

Figure 2: Temps d'exécution moyen du tri par insertion en fonction de la taille du tableau

Comme le montre la figure ci-dessus, le temps d'exécution du tri par insertion reste négligeable pour de petits tableaux (jusqu'à 10 000 éléments). Cependant, au-delà, la courbe s'envole de manière quadratique pour atteindre plus de 2000 ms à 50 000 éléments. Cela illustre parfaitement sa complexité théorique en O(n^2) et montre ses limites sur de grands jeux de données.

5.3 Les Tris Rapides : Fusion vs Dénombrement

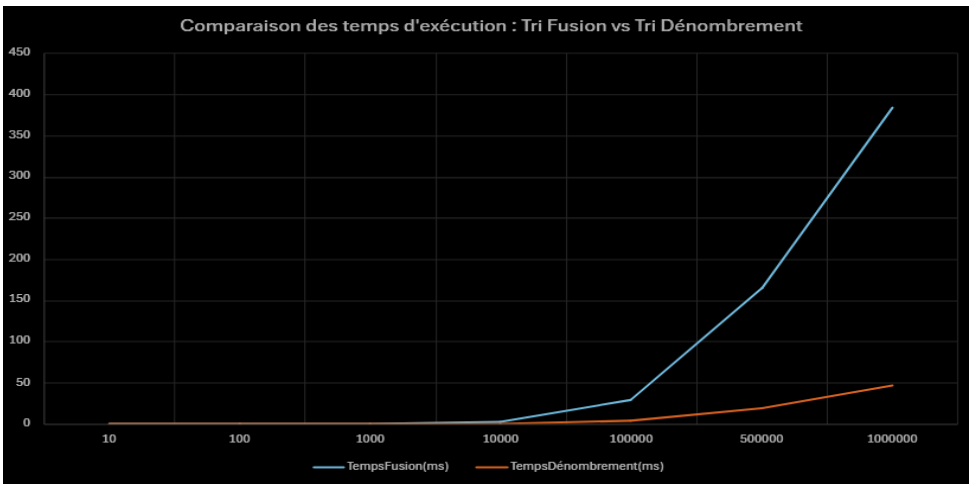

Figure 3: Comparaison des temps d'exécution : Tri Fusion contre Tri par Dénombrement

Face aux limites de l'insertion, nous avons confronté nos deux algorithmes les plus rapides. Le tri par dénombrement affiche un temps d'exécution extrêmement faible (proche de zéro), confirmant son incroyable efficacité due à sa complexité linéaire O(n+k). De son côté, le tri fusion montre une croissance très contrôlée, propre à sa complexité en O(n log n), et reste très performant même pour d'énormes quantités de données.

5.4 Comparaison globale des trois tris

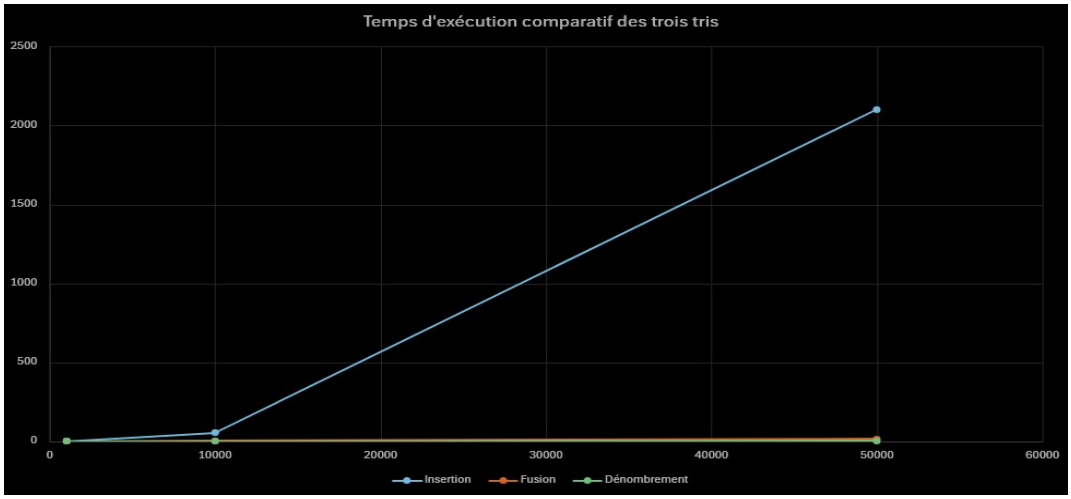

Figure 4: Comparaison globale des temps d'exécution des trois algorithmes de tri

Sur ce dernier graphique regroupant les trois algorithmes, la fracture de performance est évidente. Sur une échelle commune allant jusqu'à 50 000 éléments, la courbe du tri par insertion écrase complètement les deux autres algorithmes, qui semblent confondus avec l'axe des abscisses (restant à 0 ms). Cela prouve sans appel qu'un tri en O(n \log n) ou O(n) devient indispensable dès que la taille des données devient conséquente.

6. Conclusion

Au terme de ce projet, nous avons pu confronter la théorie algorithmique à la pratique de la programmation. L'étude du tri par insertion nous a montré que si une optimisation comme la recherche dichotomique permet de réduire drastiquement le nombre de comparaisons, elle ne change pas la complexité temporelle globale en O(n^2) à cause des opérations de décalage en mémoire qui restent nécessaires.

Ensuite, la comparaison des temps d'exécution a mis en évidence le fossé de performances entre les différentes classes de complexité. Le tri par dénombrement s'avère redoutable avec une exécution quasi instantanée sur nos jeux de test grâce à sa complexité linéaire O(n+k). Cependant, il reste contraint par l'espace mémoire si les valeurs à trier sont très grandes. Le tri fusion, quant à lui, confirme son statut d'algorithme polyvalent et très robuste, gérant de grands volumes de données avec une complexité garantie en O(n log n).

En définitive, ce travail nous a permis de comprendre qu'il n'existe pas de "meilleur" algorithme absolu, mais plutôt des algorithmes plus ou moins adaptés en fonction de la taille des données et des contraintes matérielles.